Statistical Power

Ted Ellsworth, PhD

2026-08-25

Free statistical consulting for Yale researchers

Study design · analysis · visualization · writing.

Walk-ins, appointments, and workshops — free for all Yale affiliates.

library.yale.edu/statlab

Introduction

A committee, a reviewer, an IRB, or a funder asks: **how many subjects do you need?**

What you may have been told	What we will actually do
Power is a number you look up once	Power is a property of a design you can change
Power is about sample size	Sample size is one of three levers
There is a formula, so use it	The formula describes one study. Code describes yours.

Schedule

Min	Block	Who is working
15	What power is, and what it looks like	Lecture
15	The formula, worked by hand, and the effect it cannot see	Work
15	**Your turn 1:** a tutoring study, checked in the EGAP app	**You work**
10	Where the numbers come from: guessing well	Discussion
5	*Break*	—
15	Power in Python: one formula, a grid of assumptions	Demo
15	**Your turn 2:** build the sweep, find the N you need	**You code**
15	When the formula stops describing your study: simulation	Demo
5	What voids your calculation	Lecture
10	**Your turn 3:** your own design, on the scaffold	**You code**

Learning Objectives

By the end of the session you should be able to:

You will be able to	You will show it by
Name the three ingredients of power and **identify** which you control	Reading a design and saying which lever is available
Compute power for a two-arm difference in means, and the **smallest effect** a design can see	Doing it by hand, then reproducing it in the EGAP calculator
Write the formula as a Python function and **sweep** it over a grid of sample sizes and effects	Returning the smallest N that reaches 80% power
Defend the effect size you assumed, naming its source	Reporting power as a *range*, not one number
Diagnose when the closed form no longer describes your study	Pointing to the assumption your design breaks
Simulate power for a design the formula cannot handle	Generating data from your estimand, fitting your estimator, counting rejections

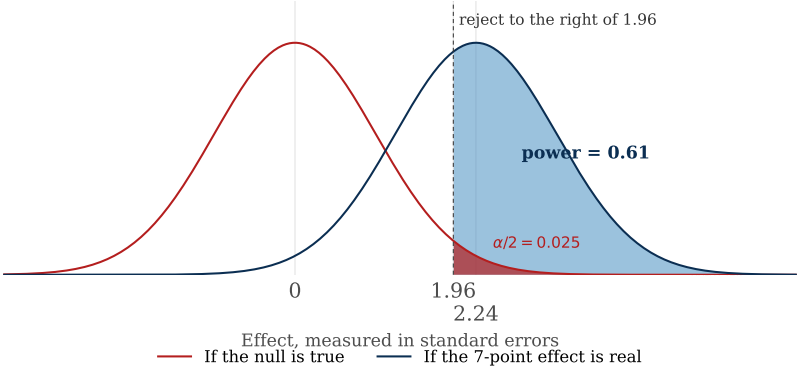
Type I and II Errors

- Most diagnostic statistics seek to identify two types of errors:
  - *Type I Errors* (α) refer to the probability that you will claim an effect is *present* when the **null hypothesis is true**.
  - *Type II Errors* (β) refer to the probability that you will claim an effect is *absent* when the **null hypothesis is false**.
- Power is what is left over from a Type II error: power $= 1 - \beta$.

What is Statistical Power?

- Less formally: power is a study's ability to *distinguish signal from noise.*
  - The *signal* is the real effect.
  - The *noise* is everything else (e.g., sampling variability, measurement error, differences between units, etc.)

What Power Looks Like



Red is the study you would run if there were no effect; the shaded sliver past 1.96 is α — the false positives you agreed to tolerate. **Blue** is the study you expect if the effect is real; **the shaded area past the same line is power**.

Type II Error Tolerance

- When conducting power analysis, our goal is to derive the probability of *detecting* a positive effect when the null hypothesis is false.
- Many studies use power $\geq 80\%$ (equivalently, $\beta \leq 0.20$) as a threshold:
  - Implies that there is a 20% chance that you will *not* find an effect that is present.
  - Better to treat as a heuristic, rather than an explicit benchmark.

Core Notation

In order to estimate statistical power, we need four qualities:

Symbol	Quality	What it is
N	Sample size	How many subjects you collect in total.
$\mu_t - \mu_c$	Effect size	The true difference between the treatment and control means: the *signal*.
σ	Standard deviation	How much outcomes vary from one another: the *noise*.
α	Significance level	The Type I error rate you are willing to tolerate; it sets the evidence threshold.

Note we will use $\mu_t - \mu_c$ throughout to denote effect size. Where space is tight — in code, and in the appendix — we abbreviate it δ.

The Three Ingredients of Statistical Power

Ingredient	Symbol	Control?	What it does
Strength of the treatment	$\mu_t - \mu_c$	Partly	Dose, framing, and duration are yours to set; a stronger treatment is easier to detect.
Background noise	σ	Partly	Noisier outcomes bury the signal, but better measurement and covariates quiet it.
Experimental design	N, α	**Yes**	Sample size, randomization scheme, and covariate adjustment.

Key Formula for Calculating Power

$$\text{power} \;=\; \Phi\left(\frac{|\mu_t - \mu_c|\sqrt{N}}{2\sigma} \;-\; \Phi^{-1}\left(1 - \frac{\alpha}{2}\right)\right)$$

Symbol	Quality	What it is
Φ	Normal CDF	The CDF of the standard normal distribution.
Φ^{-1}	Inverse normal CDF	Its inverse; returns the critical value for a given tail probability.

Source: Coppock, *10 Things to Know About Statistical Power* (EGAP). EGAP writes this quantity as β; the dominant convention reserves β for the Type II error rate, so that power $= 1 - \beta$. We follow the latter.

A Running Example: Does the Message Move Opinion?

You are fielding a survey experiment with **164 respondents**, randomly assigned 50/50 to read a short informational message (treatment) or no message (control). Support for the policy is measured on a 0–100 feeling thermometer, and a pilot study supplies your assumptions.

Symbol	Value	Where it comes from
N	164	Total respondents (82 per arm).
μ_t	57	Assumed mean support under treatment.
μ_c	50	Control mean, from the pilot.
σ	20	Standard deviation, from the pilot.
α	0.05	Two-sided, by convention.

Assumed effect: $\mu_t - \mu_c = 7$ points, or 0.35σ. **Can we detect a 7-point effect with 164 respondents?**

Step 1: Do the Arithmetic

$$\text{power} = \Phi\left(\frac{|\mu_t - \mu_c|\sqrt{N}}{2\sigma} - \Phi^{-1}\left(1 - \frac{\alpha}{2}\right)\right) \qquad \text{the formula}$$

$$= \Phi\left(\frac{7\sqrt{164}}{2(20)} - 1.96\right) \qquad \text{plug in;} \quad \Phi^{-1}(0.975) = 1.96$$

$$= \Phi(\mathbf{2.24} - 1.96) = \Phi(\mathbf{0.28}) \qquad 7 \times 12.81/40 = 2.24$$

$\underbrace{\mathbf{2.24}}$	$\underbrace{\mathbf{1.96}}$	$\underbrace{\mathbf{0.28}}$
what you expect	what you need	all the room you have

A thin margin means an unreliable study. One lookup turns $\Phi(0.28)$ into a number. (Where does 1.96 come from? Running the z-table *backwards* — see the appendix.)

Step 2: Read the Answer Off the Table

$$\text{power} = \Phi(\mathbf{0.28})$$

Split the z-score across the two margins of the table:

- **0.2** selects the **row**
- **0.08** selects the **column**

The cell where they meet is the area to the *left* of z, which is exactly the power.

z	.00	.01	.02	.03	.04	.05	.06	.07	**.08**	.09
0.0	.5000	.5040	.5080	.5120	.5160	.5199	.5239	.5279	.5319	.5359
0.1	.5398	.5438	.5478	.5517	.5557	.5596	.5636	.5675	.5714	.5753
0.2	.5793	.5832	.5871	.5910	.5948	.5987	.6026	.6064	**.6103**	.6141
0.3	.6179	.6217	.6255	.6293	.6331	.6368	.6406	.6443	.6480	.6517
0.4	.6554	.6591	.6628	.6664	.6700	.6736	.6772	.6808	.6844	.6879
0.5	.6915	.6950	.6985	.7019	.7054	.7088	.7123	.7157	.7190	.7224

$$\text{power} = \Phi(0.28) = \mathbf{0.61}$$

The Smallest Effect This Study Can See

We now know what this design does with a 7-point effect. The next question is what it *can* do: fix power at 80% and solve the same formula for the effect instead of for power.

$$\text{MDE} = \underbrace{\left(\Phi^{-1}\left(1-\tfrac{\alpha}{2}\right)+\Phi^{-1}(0.80)\right)}_{1.96+0.84=2.80} \times \underbrace{\frac{2\sigma}{\sqrt{N}}}_{\text{standard error}}$$

N	Smallest detectable effect	In standard deviations
164	**8.75 points**	**0.44σ**
257	6.99	0.35σ
400	5.60	0.28σ
800	3.96	0.20σ
1396	3.00	0.15σ

Pilot assumes $\mu_t - \mu_c = \mathbf{7}$. This design detects **8.75** and up.

Check: Put 8.75 Back Through the Formula

$$\text{power} = \Phi\left(\frac{8.75\sqrt{164}}{2(20)} - 1.96\right) \qquad \text{the MDE in place of the pilot's effect}$$

$$= \Phi(\mathbf{2.80} - 1.96) = \Phi(\mathbf{0.84}) \qquad 8.75 \times 12.81/40 = 2.80$$

$$= \mathbf{0.80} \qquad \text{row 0.8, column .04} \rightarrow .7995$$

Pin these three	Get
$N = 164, \quad \sigma = 20, \quad \text{effect} = 7$	power = 0.61
$N = 164, \quad \sigma = 20, \quad \text{power} = 0.80$	effect = 8.75

Your Turn: Is the Tutoring Program Worth Studying?

A colleague is evaluating a new tutoring program. **400 students** will be randomly assigned, 200 to tutoring and 200 to business as usual, with final exam score as the outcome. Past cohorts averaged **72** points with a standard deviation of **10**, and the program is expected to add **3** points.

What is the power of this design?

Symbol	Value
N	400
μ_t	75
μ_c	72
σ	10
α	0.05, two-sided

The formula and the z-table are on your **handout**.

Work it in three moves:

1. Compute what you **expect**
2. Subtract what you **need**
3. Look the difference up

Your Turn: The Solution

$$\text{power} = \Phi\left(\frac{|75-72|\sqrt{400}}{2(10)} - \Phi^{-1}\left(1-\tfrac{0.05}{2}\right)\right)$$ **Step 1:** plug in

$$= \Phi\left(\frac{3\times 20}{20} - 1.96\right)$$ **Step 2:** $\sqrt{400}=20$; $\Phi^{-1}(0.975)=1.96$

$$= \Phi(3.00-1.96) = \Phi(\mathbf{1.04})$$ **Step 3:** simplify

$$= \mathbf{0.85}$$ **Step 4:** row 1.0, column .04 $\rightarrow$ .8508

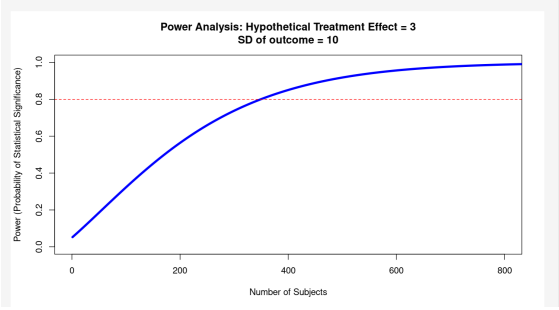
Check Your Work: The EGAP Power Calculator

https://egap.shinyapps.io/power-app/

Our symbol	Field in the app	Enter
$\mu_t - \mu_c$	Treatment Effect Size	3
σ	Standard Deviation of Outcome Variable	10
α	Significance Level	0.05
	Power Target	0.8
	Maximum Number of Subjects	800

Leave *Clustered Design* and *Binary Dependent Variable* unchecked.

Reading the Power Curve



- The x-axis is **total sample size**; the y-axis is **power**.
- The blue curve traces power as N grows, holding the effect at 3 and σ at 10.
- The red dashed line marks the **80% target**. Where the curve crosses it is the N you need (here **349**).
- Our $N = 400$ sits to the *right* of that crossing. Reading up from 400 gives $\approx$ **0.85**.
- The curve **flattens**: past $\sim$600 suggests diminishing returns at larger N.

Where to Situate Power Analysis in Your Research

- Power analysis is tricky because we need information about our data to derive power (e.g., σ, $\mu_t - \mu_c$).
- The problem is power calculations are most useful *before* we've collected our data!
- In order to do power analysis we need to do some educated guessing:
  - How big an effect should I anticipate?
  - How many people can I realistically afford to treat?
  - How many subjects will answer my survey?

Conjecture in Power Analysis

If power analysis relies on educated guesses, what can we build those guesses from?

The guess	What you can build it from
How big an effect should I anticipate? $\mu_t - \mu_c, \quad \sigma$	
How many people can I realistically afford to treat? N	
How many subjects will answer my survey? N realized	

Conjecture in Power Analysis

If power analysis relies on educated guesses, what can we build those guesses from?

The guess	**What you can build it from**
How big an effect should I anticipate? $\mu_t - \mu_c, \quad \sigma$	• Published effects, discounted for publication bias • A pilot or baseline wave: gives you σ directly • The *smallest* effect big enough to matter
How many people can I realistically afford to treat? N	
How many subjects will answer my survey? N realized	

Conjecture in Power Analysis

If power analysis relies on educated guesses, what can we build those guesses from?

The guess	**What you can build it from**
How big an effect should I anticipate? $\mu_t - \mu_c, \quad \sigma$	• Published effects, discounted for publication bias • A pilot or baseline wave: gives you σ directly • The *smallest* effect big enough to matter
How many people can I realistically afford to treat? N	• Budget divided by cost per subject • Field period, staff time, partner capacity • Size of the sampling frame you can actually reach
How many subjects will answer my survey? N realized	

Conjecture in Power Analysis

If power analysis relies on educated guesses, what can we build those guesses from?

The guess	**What you can build it from**
How big an effect should I anticipate? $\mu_t - \mu_c, \quad \sigma$	• Published effects, discounted for publication bias • A pilot or baseline wave: gives you σ directly • The *smallest* effect big enough to matter
How many people can I realistically afford to treat? N	• Budget divided by cost per subject • Field period, staff time, partner capacity • Size of the sampling frame you can actually reach
How many subjects will answer my survey? N realized	• Response rates from earlier waves or comparable surveys • Attrition rates in similar designs • Power the *analyzed* N, not the invited N

Conjecture in Power Analysis

If power analysis relies on educated guesses, what can we build those guesses from?

The guess	**What you can build it from**
How big an effect should I anticipate? $\mu_t - \mu_c, \quad \sigma$	• Published effects, discounted for publication bias • A pilot or baseline wave: gives you σ directly • The *smallest* effect big enough to matter
How many people can I realistically afford to treat? N	• Budget divided by cost per subject • Field period, staff time, partner capacity • Size of the sampling frame you can actually reach
How many subjects will answer my survey? N realized	• Response rates from earlier waves or comparable surveys • Attrition rates in similar designs • Power the *analyzed* N, not the invited N

Every one of these is a **range**, not a number. *So which number do you use?*

What If We Don't Know the Effect?

Back to the survey experiment. The pilot fixed everything *except* for the anticipated effect size.

Fixed by the pilot	
μ_c	50
σ	20
α	0.05
target power	0.80
$\mu_t - \mu_c$	?
N	?

Where an effect could come from	$\mu_t - \mu_c$
Our own pilot	7
A published study of a similar message	5
That published effect, discounted for publication bias	3

The Formula, in Code

$$\text{power} = \Phi\left(\frac{|\mu_t - \mu_c|\sqrt{N}}{2\sigma} - \Phi^{-1}\left(1 - \frac{\alpha}{2}\right)\right)$$

```
def power_calc(mu_t, mu_c, sigma, N, alpha=0.05):
    z  = np.abs(mu_t - mu_c) * np.sqrt(N) / (2 * sigma)
    cv = norm.ppf(1 - alpha / 2)
    return norm.cdf(z - cv) + norm.cdf(-z - cv)   # both rejection tails
```

Φ = `norm.cdf` $\qquad$ Φ^{-1} = `norm.ppf`

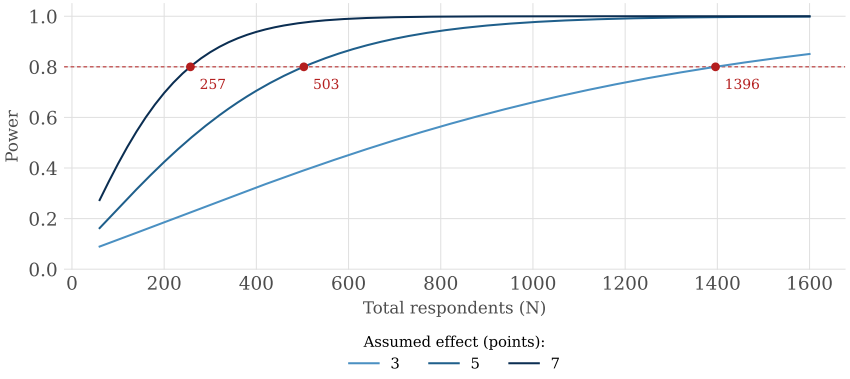
Power Across a Range of N

```
grid = pd.merge(pd.DataFrame({"N":     np.arange(60, 1601, 10)}),
                pd.DataFrame({"delta": [3, 5, 7]}), how="cross")

grid["power"] = power_calc(mu_t  = 50 + grid["delta"],
                           mu_c  = 50,
                           sigma = 20,
                           N     = grid["N"])

grid.head(3)
```

	N	delta	power
0	60	3	0.089473
1	60	5	0.162372
2	60	7	0.273240

One formula, three assumptions, 465 rows. Now plot `power` against `N`.

How Many Respondents Do We Need?



Your Turn: Build the Sweep in Python

Your department is considering a six-week **peer-mentoring program** for first-year students, and can randomize students into it or onto a waitlist. The outcome is end-of-year GPA. The registrar tells you first-year GPA averages **3.05**, with a standard deviation of **0.45**.

Given	
μ_c	3.05
σ	0.45
α	0.05

You decide:

- Which GPA gains are plausible?
- How many students could you enroll?

Your Turn: One Solution

```
grid = pd.merge(pd.DataFrame({"N":     np.arange(50, 1001, 10)}),
                pd.DataFrame({"delta": [0.10, 0.15, 0.20]}), how="cross")
grid["power"] = power_calc(mu_t = 3.05 + grid["delta"], mu_c = 3.05,
                           sigma = 0.45, N = grid["N"])

for d in grid["delta"].unique():
    ok = grid.loc[(grid["delta"] == d) & (grid["power"] >= 0.80), "N"]
    print("effect", d, "GPA points -> N =", ok.min())
```

```
effect 0.1 GPA points -> N = 640
effect 0.15 GPA points -> N = 290
effect 0.2 GPA points -> N = 160
```

Your effects and N range will differ. The *structure* is the answer: one grid, one call, one threshold.

Where the Formula Runs Out

The closed form is not general. It is designed for difference of means testing; if we want to use a more complex model, we need a different approach:

It assumes	So it cannot handle
Two arms, split 50/50	Three arms, unequal allocation
One continuous, normal outcome	Binary, counts, skewed scales
A simple difference in means	Covariates, fixed effects, IV, mixed models
Independent observations	Clustered or repeated measures
Everyone recruited is analysed	Attrition, non-response

Power with Monte Carlo Simulation

1. **Invent a world** where your assumptions are literally true: $\mu_t - \mu_c = 7$, $\sigma = 20$, $N = 164$.
2. **Draw one fake study** from that world and run *the analysis you actually plan to run.*
3. **Repeat** a few thousand times. Power is the share of those studies where $p < 0.05$.

What Simulation Adds to the Sweep

	Sweep	**Simulation**
What varies	Inputs to one formula	Inputs, *and* the design and estimator
Power comes from	A closed-form CDF	The share of fake studies that reject
Cost	Exact, instant	Approximate, seconds to minutes
Works for	Two arms, means, complete data	Anything you can write code for
You also get	One number	The whole distribution of estimates and p-values

They are not rivals. The sweep is the **outer loop**; simulation changes what fills each cell.

Estimand and Estimator

	What it is	Examples
Estimand	The quantity you want to learn. A property of the *world*.	The ATE; the effect among women; an interaction; a LATE
Estimator	The procedure you run on data. A property of your *code*.	Difference in means; OLS with covariates; 2SLS; a mixed model

Anatomy of One Simulation

Before automating anything, run the study **once**.

```
rng = np.random.default_rng(26)

def ols(formula, **cols):   # R's lm() reads from the calling frame; statsmodels
    return smf.ols(formula, pd.DataFrame(cols)).fit()      # wants a DataFrame
def one_study():
    z = rng.permutation(np.repeat([0, 1], 82))  # RANDOMLY assign the message
    y = rng.normal(50 + 7 * z, 20)              # ESTIMAND: the truth is 7
    return ols("y ~ z", y=y, z=z).pvalues["z"]  # ESTIMATOR: what you run

[round(one_study(), 4) for _ in range(3)]
```

```
[0.0158, 0.4322, 0.236]
```

Power is just the share of these that land under 0.05.

How Many Simulations Is Enough?

A simulated power estimate is itself an estimate. Its standard error is $\sqrt{\text{power}(1-\text{power})/S}$.

S	$\pm$ **at power** $= 0.80$	**Good for**
200	0.028	A first look, nothing more
1,000	0.013	Exploring designs
2,000	0.009	Most teaching and prototyping
10,000	0.004	A number going in a grant or pre-registration

The cheap check: rerun with a different seed. If the answer moves more than you can live with, raise S.

Building a Monte Carlo simulation for Power

Keep the sweep; replace what fills the column. Run the study thousands of times and count.

```
def sim_power(N, tau, sigma, S=2000):
    p = np.empty(S)
    for i in range(S):
        z = rng.permutation(np.resize([0, 1], N))
        y = rng.normal(50 + tau * z, sigma)          # the ESTIMAND
        p[i] = ols("y ~ z", y=y, z=z).pvalues["z"]  # the ESTIMATOR
    return np.mean(p < 0.05)

rng = np.random.default_rng(11)
pd.Series({"simulation": sim_power(164, 7, 20),
           "formula":    power_calc(57, 50, 20, 164)}).round(4)
```

```
simulation    0.6050
formula       0.6107
dtype: float64
```

What If I Change My Analysis Plan?

In the survey experiment you planned to compare mean thermometer scores. Now you decide to fit a regression that adjusts for each respondent's *baseline* thermometer reading, taken before the message.

	What you planned	**What you now plan**
The code	`ttest_ind(y[z==1], y[z==0])`	`ols("y ~ z + x")`
Estimand	The ATE of the message	**the same ATE**
Estimator	Difference in means	OLS with a covariate
Does the closed form describe it?	Yes	**No**

You changed **how you will answer**, not **what you are asking**.

The baseline reading `x` is measured before treatment, so it is a *precision covariate*, not a confounder; it correlates about 0.6 with the outcome. Does adjusting help or hurt? **Simulate it.**

Branch 1: Same Estimand, Better Estimator

```
def sim_cov(adjust, N=164, S=2000, tau=7):
    p = np.empty(S)
    for i in range(S):
        z = rng.permutation(np.resize([0, 1], N)); x = rng.normal(0, 1, N)  # x = baseline
        y = 50 + tau * z + 12 * x + rng.normal(0, 16, N)      # cor(x, y) = 0.6
        p[i] = ols("y ~ z + x" if adjust else "y ~ z",
                   y=y, z=z, x=x).pvalues["z"]
    return np.mean(p < 0.05)

rng = np.random.default_rng(11)
pd.Series({"unadjusted": sim_cov(False), "adjusted": sim_cov(True)})
```

```
unadjusted    0.596
adjusted      0.782
dtype: float64
```

Nearly 80% power at the same N: adjusting drops σ from 20 to 16, at **zero** extra cost in respondents.

The Smallest Effect the Regression Can See

The formula does not describe `ols("y ~ z + x")`, so fix power at 80% and sweep the **effect** by simulation, holding N at 164.

```
rng = np.random.default_rng(5); taus = np.arange(5, 10)   # candidate effects
pw  = np.array([sim_cov(True, S=1000, tau=t) for t in taus])
def mde(p, tau, S=1000, target=0.80):                  # fit, then invert
    succ = np.round(np.asarray(p) * S)
    X = sm.add_constant(np.log(np.asarray(tau, float)))
    b = sm.GLM(np.column_stack([succ, S - succ]), X,
               family=sm.families.Binomial()).fit().params
    return np.exp((logit(target) - b[0]) / b[1])
print(pd.DataFrame([dict(zip(taus, pw.round(2)),
                         MDE=round(mde(pw, taus), 2))]).to_string(index=False))
```

```
   5    6   7    8    9  MDE
0.53 0.66 0.8 0.88 0.95  6.9
```

Adjusting moves the MDE from **8.75** points to **6.9**. The closed form with $\sigma = 16$ gives 7.0.

What If x Is a Confounder Instead?

Same covariate, but z is now **not randomized**: higher x makes treatment likelier. Watch the rejection rate.

```
rng = np.random.default_rng(3)
def sim_obs(adjust, N=164, S=2000):
    est, pv = np.empty(S), np.empty(S)
    for i in range(S):
        x = rng.normal(0, 1, N); z = rng.binomial(1, expit(1.2 * x))  # z DEPENDS on x
        y = 50 + 7 * z + 12 * x + rng.normal(0, 16, N)                # truth is still 7
        f = ols("y ~ z + x" if adjust else "y ~ z", y=y, z=z, x=x)
        est[i], pv[i] = f.params["z"], f.pvalues["z"]
    return est.mean(), np.mean(pv < 0.05)
obs = pd.DataFrame([sim_obs(False), sim_obs(True)], columns=["estimate", "rejection"],
                   index=["y ~ z", "y ~ z + x"]); print(obs.round(3).to_string())
```

```
           estimate  rejection
y ~ z        18.048      1.000
y ~ z + x     6.997      0.688
```

The unadjusted model rejects **100%** of the time. That is *not* power — the estimate is centred on 18.0 when the truth is 7. A rejection rate is only power once the estimator is unbiased.

Branch 2: Same Study, Different Estimand

The message moves $g = 1$ by 10 points and $g = 0$ by 4. Average effect still 7; the **interaction** is 6.

```
rng = np.random.default_rng(11)
ate, inter = np.empty(2000), np.empty(2000)
for i in range(2000):
    z = rng.permutation(np.repeat([0, 1], 82)); g = np.tile([0, 1], 82)
    y = rng.normal(50 + 4 * z + 6 * z * g + 2 * g, 20)
    ate[i]   = ols("y ~ z",     y=y, z=z, g=g).pvalues["z"]
    inter[i] = ols("y ~ z * g", y=y, z=z, g=g).pvalues["z:g"]

pd.Series({"ate": np.mean(ate < 0.05), "int": np.mean(inter < 0.05)})
```

```
ate    0.6170
int    0.1535
dtype: float64
```

Nothing changed but the **question**. An interaction carries twice the standard error, so it behaves like a main effect with σ **doubled**: 80% power needs $\approx$**1400** respondents, against 257 for the ATE.

Power Is a Property of the Triple

Choice	What it is	What it did here
Estimand	The quantity you are after: ATE, interaction, subgroup, LATE	$0.61 \rightarrow 0.15$
Estimator	The model you will actually fit	$0.61 \rightarrow 0.78$
Design	N, allocation, blocking, what gets randomized	the lever fully in your hands

Power is not a property of your sample size. It is a property of all three together.

What is my estimand, what estimator will I actually run, and what does *that pair* need?

What Voids Your Calculation

Everything so far assumed you randomize individuals, one at a time, on a continuous outcome, and analyse everyone you recruited. Break any of those and the number you just computed is wrong — usually by a lot.

If your design has...	Your N is wrong by	What that looked like here
Groups randomized, not individuals	$\times$ DEFF $= 1 + (m-1)\rho$	283 $\rightarrow$ **1,302** students; analysed naively, a **36%** false-positive rate
Take-up below 100%	$\div$ take-up^2	50% take-up $\rightarrow$ **4**$\times$ the sample: 257 $\rightarrow$ 1,028
Attrition	$\div$ (1 $-$ loss)	20% loss $\rightarrow$ recruit **322** to analyse 257
A binary outcome	$\sigma = \sqrt{p(1-p)}$	The same 164 people: 61% power continuous, **36%** for 30%$\rightarrow$42%
A subgroup or interaction	$\approx$ 4$\times$	257 $\rightarrow$ **$\sim$1,400**

Your Turn: Simulate a Design of Your Own

Take a study you are actually planning, or one you have just read. Before writing any code, answer three questions:

1. **What is my estimand?** An average effect, a difference between groups, an effect for compliers only?
2. **What estimator will I actually run?** A difference in means, OLS with controls, something with clustered errors?
3. **What must I assume to simulate it?** A baseline mean, an outcome standard deviation, a plausible effect.

Then fill in the scaffold on the next slide and sweep it over N.

Your Turn: A Scaffold

```
def one_study(N):
    z = rng.permutation(np.resize([0, 1], N))  # 1. ASSIGN: randomize treatment

    y = rng.normal(???, ???, N)          # 2. GENERATE: your ESTIMAND

    d = pd.DataFrame({"y": y, "z": z})
    return smf.ols("y ~ z", d).fit().pvalues["z"]   # 3. ANALYSE: your ESTIMATOR

def power_for(N, S=1000):
    return np.mean(np.array([one_study(N) for _ in range(S)]) < 0.05)

[power_for(N) for N in range(100, 601, 50)]
```

Change line 2 and you have changed your estimand. Change line 3 and you have changed your estimator. Everything else stays put.

Appendix: Where 1.96 Comes From

$$\Phi^{-1}(\mathbf{0.975})$$

Φ^{-1} runs the table **backwards**. You are handed an area and must find the z that produces it:

1. Hunt .9750 in the **body**
2. Read off its **margins**

Row 1.9 plus column .06 gives the critical value you already know: **1.96**.

z	.00	.01	.02	.03	.04	.05	**.06**	.07	.08	.09
1.6	.9452	.9463	.9474	.9484	.9495	.9505	.9515	.9525	.9535	.9545
1.7	.9554	.9564	.9573	.9582	.9591	.9599	.9608	.9616	.9625	.9633
1.8	.9641	.9649	.9656	.9664	.9671	.9678	.9686	.9693	.9699	.9706
1.9	.9713	.9719	.9726	.9732	.9738	.9744	**.9750**	.9756	.9761	.9767
2.0	.9772	.9778	.9783	.9788	.9793	.9798	.9803	.9808	.9812	.9817
2.1	.9821	.9826	.9830	.9834	.9838	.9842	.9846	.9850	.9854	.9857

Start *inside* the table and read outward: the mirror image of Step 2.

$\Phi^{-1}(0.975) = \mathbf{1.96}$: the bar the study has to clear.

Appendix: Two Loops, Not One

The sweep and the simulation stack. Neither replaces the other.

	What it loops over	Question it answers
Outer `merge(how="cross")`	The designs you are considering: each N, each estimator	"Which study should I run?"
Inner `for i in range(S)`	Random draws from one fixed design	"How often does *this* study win?"

```
grid = pd.merge(pd.DataFrame({"N": np.arange(120, 301, 30)}),      # OUTER
                pd.DataFrame({"adjust": [False, True]}), how="cross")
grid["power"] = [sim_cov(a, n)                                     # INNER runs inside
                 for n, a in zip(grid["N"], grid["adjust"])]
```

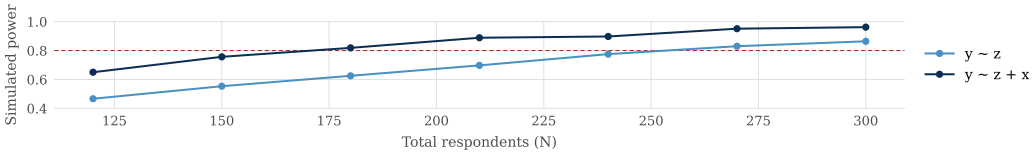
Appendix: Sweeping the Simulation

```
rng = np.random.default_rng(4); Ns = np.arange(120, 301, 30)
sweep_sim = pd.DataFrame({"N": Ns,
    "unadjusted": [sim_cov(False, n, S=1500) for n in Ns],
    "adjusted":   [sim_cov(True,  n, S=1500) for n in Ns]})
print(sweep_sim.round(3).to_string(index=False))
```

```
  N  unadjusted  adjusted
120       0.467     0.650
150       0.553     0.757
180       0.625     0.819
210       0.697     0.889
240       0.775     0.897
270       0.830     0.951
300       0.864     0.962
```

Unadjusted crosses 80% between 240 and 270; adjusted, between 150 and 180. The closed form puts the two at **257** and **165**.

Appendix: Finding the Cutoff by Simulation




```
def cutoff(p, N, S=1500, target=0.80):        # fit, then invert
    succ = np.round(np.asarray(p) * S)
    X = sm.add_constant(np.log(np.asarray(N, float)))
    b = sm.GLM(np.column_stack([succ, S - succ]), X,
               family=sm.families.Binomial()).fit().params
    return np.exp((logit(target) - b[0]) / b[1])
{"adjusted":   round(cutoff(sweep_sim["adjusted"],   sweep_sim["N"])),
 "unadjusted": round(cutoff(sweep_sim["unadjusted"], sweep_sim["N"]))}
```

```
{'adjusted': 164, 'unadjusted': 257}
```

Do **not** run `np.interp()` over simulated power: noise makes the curve non-monotone. Fit a monotone curve, invert that.

Appendix: The Same Covariate, Two Different Jobs

Which job `x` is doing depends entirely on how `z` was assigned.

	Randomized: x is a *precision covariate*	**Observational:** x is a *confounder*
Why include it	Optional. Randomization already gives the right answer.	Mandatory. Without it the answer is wrong.
Estimate, `y ~ z`	7.00 (truth is 7)	**180**☒ (truth is 7)
Adding x	Soaks up outcome variance ⇒ **more power**	Also soaks up *treatment* variance ⇒ **less power**
Power, `y ~ z + x`	**0.78**	06☒9

Appendix: When You Randomize Groups, Not People

Assign whole classrooms, clinics, or villages and observations within a cluster are correlated. Effective sample size shrinks by the **design effect**:

$$\text{DEFF} = 1 + (m - 1)\,\rho \qquad m = \text{cluster size}, \quad \rho = \text{intra-cluster correlation}$$

GPA study, $\delta = 0.15$	**Students needed**
Randomize individual students	283
Randomize classrooms of 25, $\rho = 0.15$ (DEFF 4.6)	**1300**

That checkbox we told you to leave unchecked in the EGAP app? *This is what it does.*

Appendix: Clustering in Simulation

The danger: the naive analysis still returns a number, and the number looks fine.

```
rng = np.random.default_rng(7)
n_clus, m = 12, 25                          # 300 students in 12 classrooms
sb, sw = 0.45 * np.sqrt(0.15), 0.45 * np.sqrt(0.85)
p = np.empty((1500, 2))
for i in range(1500):
    zc = rng.permutation(np.repeat([0, 1], n_clus // 2))  # assign CLASSROOMS
    u  = rng.normal(0, sb, n_clus)                        # classroom-level shock
    y  = rng.normal(np.repeat(3.05 + 0.15 * zc + u, m), sw)
    zi, cm = np.repeat(zc, m), y.reshape(n_clus, m).mean(axis=1)
    p[i] = (ttest_ind(y[zi == 1],  y[zi == 0],  equal_var=False).pvalue,
            ttest_ind(cm[zc == 1], cm[zc == 0], equal_var=False).pvalue)
pd.Series(dict(zip(["as_individuals", "as_clusters"], (p < 0.05).mean(axis=0))))
```

```
as_individuals    0.696000
as_clusters       0.222667
dtype: float64
```